北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

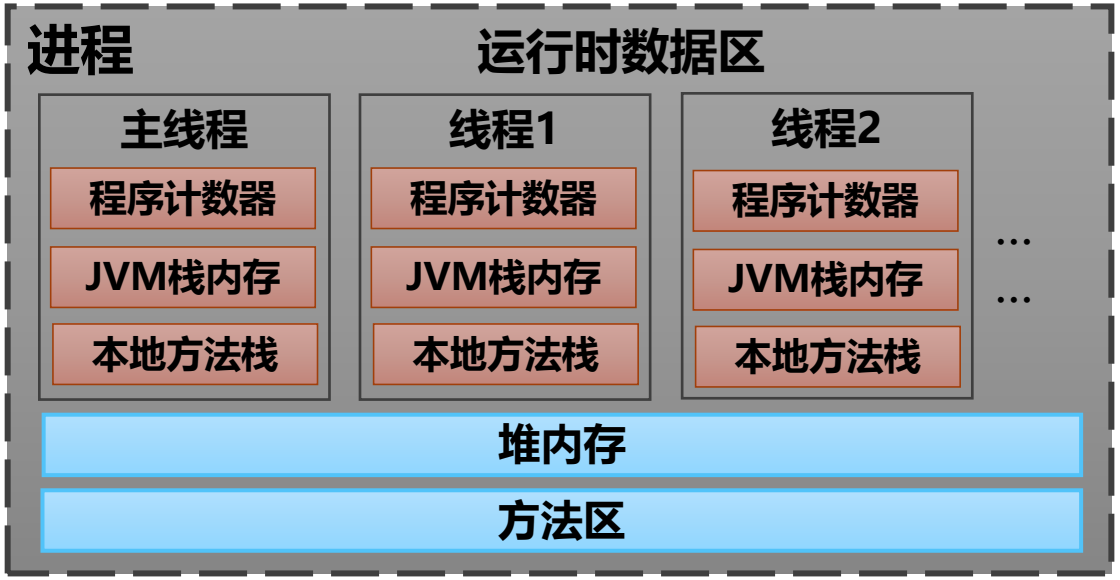
第11章 多线程

本章知识点

- 进程与线程
- 线程的创建与执行
- 线程的生命周期
  - 新建、就绪、运行、阻塞、死亡
- 线程的同步
  - synchronized
  - 同步机制
  - 同步代码块、同步方法
- 线程通信
  - Wait(),notify(),notifyAll()
  - 生产者与消费者模型

进程和线程的关系

- 线程作为处理器调度和分配的基本单位，而进程是作为拥有资源的基本单位。
- 进程是拥有资源的一个独立单位，有自己独立的地址空间；线程不拥有系统资源，但可以访问隶属于进程的资源，共享进程的地址空间



进程和线程的关系

- 线程的优势

（1）操作系统创建进程时需要为其分配独立的内存单元及分配大量的相关资源，相比而言，线程的创建简单得多。因此多线程的多任务并发比多进程的效率要高。

（2）进程之间不能共享内存，线程之间共享内存则非常容易。

（3）Java语言提供了多线程的支持，不需本地操作系统的直接参与，简化了多线程编程。

11.2 线程的创建和执行

- 线程：java.lang.Thread实例
- 创建线程的方式
  - 继承Thread类
  - 实现Runnable接口
  - 实现callable接口；

11.2.1 继承Thread类创建线程

（1）定义Thread的子类，并重写该类的run()方法。run()方法的方法体代表了线程需要完成的任务。

（2）创建线程对象。

（3）线程对象调用start()方法启动该线程。

11.2.1 继承Thread类创建线程

（1）定义Thread的子类，并重写该类的run()方法。run()方法的方法体代表了线程需要完成的任务。

（2）创建线程对象。

（3）线程对象调用start()方法启动该线程。

【例11-1】通过Thread类创建线程。

11.2.2 实现Runnable接口创建线程

1、实现Runnable接口，并重写run()方法。
run 方法是程序的执行体，即线程要做的事情

2、创建Runnable实现类的实例

3、以Runnable实现类的实例为target，创建Tread对象并调用start方法。

【例11-2】通过Runnable接口创建线程。

11.2.3 实现Callable接口创建线程

Callable接口是Runnable接口的改进版

1、Callable接口中的call方法有返回值；

2、Callable接口中的call方法声明抛出异常

创建线程的步骤：

1、实现Callable接口，并重写call()方法。

call 方法是程序的执行体，即线程要做的事情

2、创建Callable接口实现类的实例

3、将Callable接口实现类的实例包装成FutureTask对象；

3、以FutureTask对象为target，创建Tread对象并调用start方法。

注意：Callable与FutureTask的泛型要一致（指定线程执行体（call方法）的返回值）

三种创建线程方式的对比：

实现接口的方式更好，原因如下：

A。实现接口之后，线程体还可以继承其他父类；但如果已经继承Thread，不能继承其他父类。

B。实现接口方式创建的线程，可以方便地使用线程池，性能更好！

C。对于继承Thread的方式创建线程类，每个线程都需要创建一个线程类的实例，从而造成数据逻辑与业务逻辑混杂。

对于实现接口的方式，多个线程可共用同一个target，数据逻辑与业务逻辑是分离的。

继承Thread创建的好处在于：编程简单，适合初学者！

11.3 线程的状态与生命周期

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 线程由新建、就绪、运行、阻塞、死亡这些状态构成了它的生命周期，呈现了其工作的过程。

11.3.1 新建和就绪状态

创建:
刚刚创建Thread类或Thread子类的实例，此时线程只是普通Java对象。

就绪:
调用start方法之后，就进入ready。
处于就绪状态的线程，随时可能进行运行。
到底何时进入运行，取决线程调度器的调度，程序员不知道。

11.3.2 运行状态

- 运行
- 处于就绪状态的线程获得了CPU进入“运行”状态（running），开始执行。

- 操作系统大都采用抢占式的调度策略，即系统给每个可执行的线程一个时间片来处理任务，时间片用完后线程被换下。选择下一个线程时，系统会考虑线程的优先级。

- 线程将一直在运行、就绪之间切换

11.3.3 阻塞状态

- “阻塞”状态（blocked）是三种状态的组合体：
睡眠/资源阻塞/等待。

共同点：线程依然是活的，但当前缺少运行它的条件，即当前是不可运行的，如果发生某个特定的事件，它将返回runnable状态。

阻塞状态下的线程无法向下执行

11.3.3 阻塞状态

- 线程在运行状态时，遇到如下状况将会进入各种阻塞状态。
- 睡眠：线程调用sleep()方法睡眠一段时间。
- 资源阻塞：线程在等待一种资源，例如线程调用了阻塞式的I/O方法（等待输入流等），在该方法返回之前该线程被阻止；线程试图获得一个同步锁，但同步锁正被其他线程持有
- 等待：线程调用wait()方法后等候其他线程的唤醒通知。

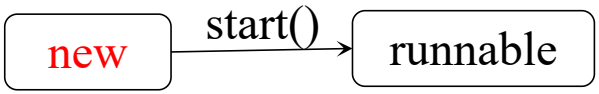
11.3.4 死亡状态

- 线程的run()方法执行完毕，线程正常结束；
- 或者线程执行过程中抛出一个未捕获的异常或错误，线程异常结束。结束后线程处于“死亡”状态（dead）。

线程状态转换图

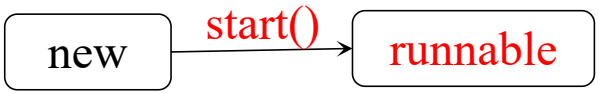

```
public class FirstThread extends Thread{
    //重写run方法
    public void run(){
        for(int i=0; i<5; i++){
            System.out.println(this.getName()+" :  "+(char)(i+'A'));
            for(int j=0; j<400000000; j++);
        }
    }
}

public class SecondThread implements Runnable{
    //重写run方法
    public void run(){
        for(int i=0; i<5; i++){
            System.out.println(Thread.currentThread().getName()
            +" :  "+(char)(i+'A'));
            for(int j=0; j<400000000; j++);
        }
    }
}
```

线程状态转换图



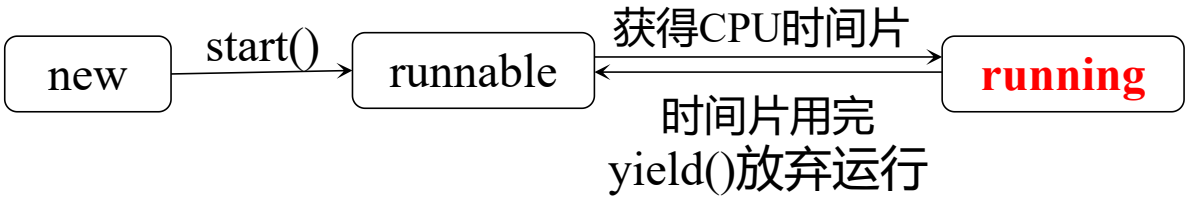

```
public static void main(String[] args) {
    Thread t1 = new FirstThread();


    Runnable target = new SecondThread();
    Thread t2 = new Thread(target);

}
```

线程状态转换图



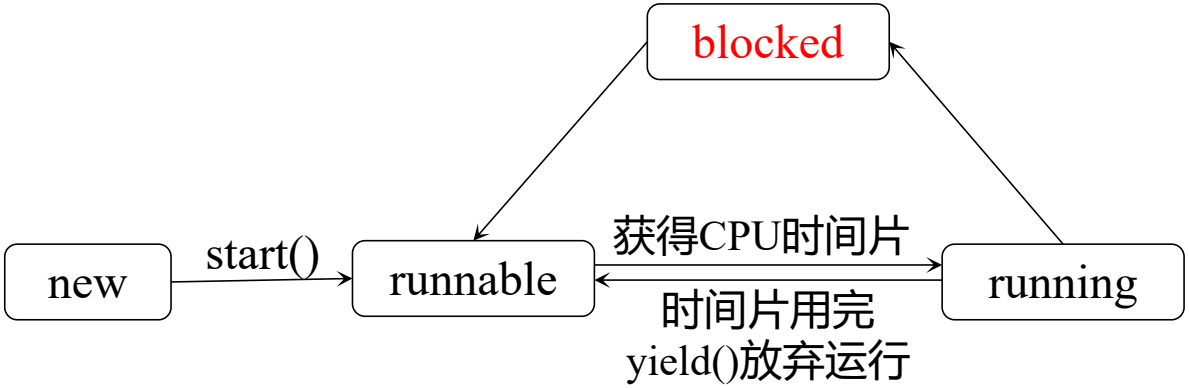

```
public static void main(String[] args) {
    Thread t1 = new FirstThread();
    t1.start();

    Runnable target = new SecondThread();
    Thread t2 = new Thread(target);
    t2.start();
}
```

线程状态转换图



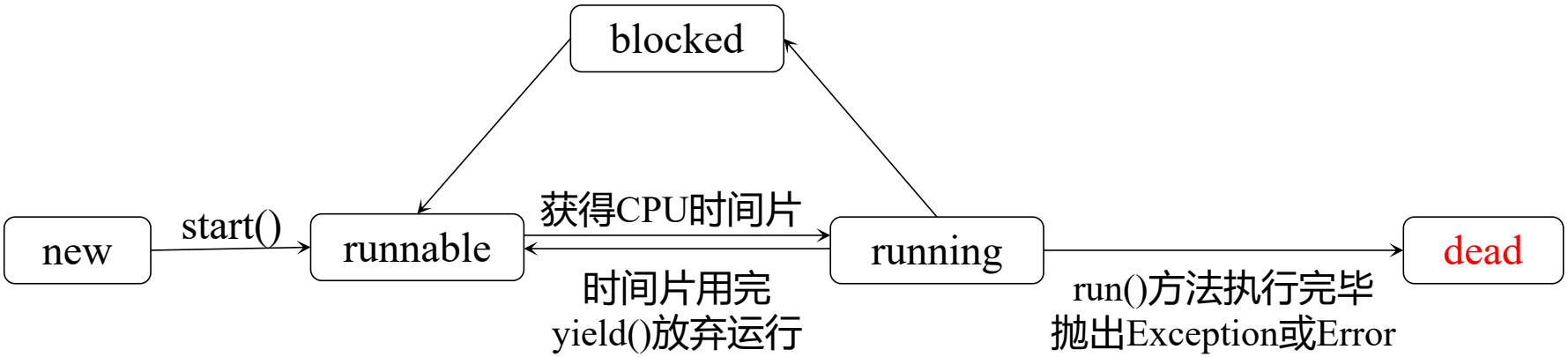
- 如果处于就绪状态的线程获得了CPU时间片，就开始执行run()方法中的线程执行体，线程进入“运行”状态。
- 除非线程的执行体特别短，在一个CPU时间片内就可以执行完毕，否则在运行过程中它将会被中断，以使其他的线程获得执行的机会。线程因失去时间片而中断时返回就绪状态。
- running状态的线程可以调用yield()方法主动放弃执行，从running转入runnable。

线程状态转换图



- “阻塞”状态是三种状态的组合体：睡眠/资源阻塞/等待。这三种状态有一个共同点：线程依然是活的，但当前缺少运行它的条件，即当前是不可运行的，如果发生某个特定的事件，它将返回就绪状态。

线程状态转换图



- 如果线程的run()方法执行完毕，线程正常结束；或者线程执行过程中抛出一个未捕获的异常或错误，线程异常结束。结束后线程处于“死亡”状态。

11.4 线程优先级与线程调度策略

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 线程调度器：JVM的一部分
  - JVM通常将Java线程直接映射为本地操作系统上的本机线程。
  - 处于runnable状态的线程被放在可运行池中，它们都有资格被调度
  - 线程调度器决定在某个时刻应该运行哪个线程。
  - 决定实际运行哪个线程是线程调度器的职权，我们无法控制线程调度器，不能要求、指定某个线程去被运行。

- 线程调度器：基于优先级的抢先式调度机制。

在设计多线程应用程序时不能依赖线程的优先级。线程调度优先级操作是没有保证的，只能将线程优先级作为一种提高程序效率的方法。

设置优先级setPriority(int);
一般使用三个优先级：MIN_PRIORITY
NORM_PRIORITY
MAX_PRIORITY

当程序需要让某个线程获得更多的执行机会时，即可将它优先级设高。
当程序需要让某个线程获得更少的执行机会时，即可将它优先级设低。

11.5.1 数据共享问题

- 【提出问题】夫妻共同进行取钱。
  - ①妻子线程首先执行，检查账户发现账户余额满足取款条件（妻子线程的步骤(1)完成），但在妻子取款之前线程被换下；
  - ②丈夫线程上来后检查账户余额，此时妻子还未取款，丈夫看到的是妻子取款前的账户余额，账户余额也可以满足他的取款要求（丈夫线程的步骤(1)完成），在丈夫取款之前线程被换下；
  - ③妻子线程换上来后接着运行，取走了账户中的全部余额（妻子线程的步骤(2)完成），妻子线程结束；
  - ④丈夫线程换上来后接着运行，因为之前丈夫线程已经确认账户有足够的余额可以支取，于是，丈夫在没有足够余额的情况下仍然进行了取款。

线程安全：在同一时刻，多条线程并发修改一份竞争资源（共享资源），此时就可能发生线程安全问题。

- 当多个线程共享同一个数据资源时，如果处理不当，很容易出现线程的安全隐患，所以多线程编程时经常需要解决线程同步问题。

```
public static void main(String[] args) {
    Runnable target = new SecondThread();

    Thread t1 = new Thread(target);
    Thread t2 = new Thread(target);

    t1.start();
    t2.start();
}
```

11.5.2 同步和锁机制

- 为了保证共享对象的线程安全，Java使用同步监视器。
  - Java使用关键字**synchonized**修饰同步代码块或同步方法，为共享资源对象加锁。
  - 任何线程进入同步代码之前首先对共享资源对象加锁，必须等到该线程离开同步块代码时，才释放对对共享资源对象的加锁。
  - 加锁后的同步代码块或同步方法，形成“原子”操作，该操作是不可分割的。
  -

11.5.2 同步和锁机制

- 线程和锁的关系
  - 锁属于对象，线程持有对象的锁可以运行同步代码；同步代码运行完毕后锁被释放。
  - 因为一个对象只有一个锁，所以当一个线程对对象加锁后，其它试图对同一个对象执行同步代码的线程，都会因获取锁不成功而进入阻塞状态，从而保证了线程安全。

锁不属于线程，而是属于对象，一个线程可以拥有多个对象的锁，而只有同一个对象的锁之间才有互斥的作用。

11.5.3 同步代码块

- 注意：同步所形成的“原子”操作会损害并发性，所以不要同步原子操作之外的其他代码。

语法：

```
synchronized(同步监视器)
{
    同步代码块

}
```

【例11-5】设计一个实现Runnable接口的线程类，输出1-5间的数字。其中，计数器变量i作为属性存在。

```
public class MyRunnable implements Runnable{
    private int i=0;  //i作为属性
    public void run(){
        while(i<5){
            i++;
            for (int j = 0; j < 20000000; j++);
            System.out.print(Thread.currentThread().getName()+" ");
            System.out.println("i="+i);
        }
    }
}
```

【例11-5】设计一个实现Runnable接口的线程类，输出1-5间的数字。其中，计数器变量i作为属性存在。

```
public class Test {
    public static void main(String[] args) {
        Runnable target = new MyRunnable();
        Thread t1 = new Thread(target, "A");
        Thread t2 = new Thread(target, "B");
        t1.start();
        t2.start();
    }
}
```

（1）状况1：i=1为什么没有被输出?

线程A

线程B

```
public void run(){
    while(i<5){
        i++;
        for (int j = 0; j < 20000000; j++);
        System.out.print(Thread.currentThread().getName());
        System.out.println("i=" + i);
    }
}
```

①i=1
时间片到被换下

② 被换上
i=2
输出：B i=2

线程A

线程B

```
public void run(){
    while(i<5){
        i++;
        for (int j = 0; j < 20000000; j++);
        System.out.print(Thread.currentThread().getName());
        System.out.println("i=" + i);
    }
}
```

① i=4
时间片到被换下

② 被换上
i=5
输出：A i=5
时间片到被换下

③ 被换上
输出：B i=5

线程A　　线程B

```
public void run(){
    while(i<5){
        i++;
        for (int j = 0; j < 20000000; j++);
        System.out.print(Thread.currentThread().getName());
        System.out.println("i=" + i);
    }
}
```

线程A　　线程B

```
public void run(){
    while(i<5){
        i++;
        for (int j = 0; j < 20000000; j++);
        System.out.print(Thread.currentThread().getName());
        System.out.println("i=" + i);
    }
}
```

同步代码块

```
synchonized (对象) {
        ……
}
```

```
public void run(){
    while(i<5){
        synchronized(this){  //加锁
            i++;
            for(int i=1; i<=10000000;i++);
            System.out.print(Thread.currentThread().getName());
            System.out.println(" i="+i);
        }

    }
}
```

```java
public class MyRunnable implements Runnable{
    private int i=0;  //i作为属性
    public void run(){
        while(i<5){
            synchronized(this){
                i++;
                for (int j=0; j<20000000; j++);
                System.out.print(Thread.currentThread().getName()+" ");
                System.out.println("i="+i);
            }
        }
    }
}
```

```java
public class Test {
    public static void main(String[] args) {
        Runnable target = new MyRunnable();
        Thread t1 = new Thread(target, "A");
        Thread t2 = new Thread(target, "B");
        t1.start();
        t2.start();
    }
}
```

（3）状况3：为什么输出i=6?

线程A

线程B

```
public void run(){
    while(i<5){
        synchronized(this){ //加锁
            i++;
            for(int i=1; i<=10000000;i++);
            System.out.print(Thread.currentThread().getName());
            System.out.println(" i="+i);
        }
    }
}
```

①i=4, 通过i<5
时间片到被换下

③ 被换上
获得锁
进入同步块
i=6
输出 A i=6
……

②被换上
加锁，进入同步块
i=5
输出： B i=5
释放锁
时间片到被换下

最终的代码

```
public class MyRunnable implements Runnable{
    private int i=0;  //i作为属性
    public void run(){
        while(i<5){
            synchronized(this){
                if(i==5) break;
                i++;
                for (int j = 0; j < 20000000; j++);
                System.out.print(Thread.currentThread().getName()+" ");
                System.out.println("i="+i);
            }
        }
    }
}
```

11.5.4 同步方法

- 如果一个方法内的所有代码组成“原子”操作，那么可以将该方法定义为同步方法，使用synchronized关键字修饰。
- 如果是实例方法，则方法的调用者（this）被加锁
- 如果是类方法，则方法的调用者（所在类）被加锁
- 无论是同步代码块还是同步方法都会导致性能的降低，所以要精确控制同步代码块的区域。

【例11-6】完成多线程环境下的银行取钱操作。

11.6 线程间的通信

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 在多线程环境中，线程之间经常需要协调通信从而共同完成一件任务。Java传统的线程通信是通过Object类中的wait()和notify()方法完成。
- Java SE5.0中增加了阻塞队列BlockingQueue等方式控制线程通信。

11.6.1 wait()和notify()方法

- Object类中提供了wait()，notify()和notifyAll()3个方法来操作线程。
- 它们只能在同步代码块或者同步方法内使用，而且只能通过进行同步控制的对象（同步监视器）来调用。

```
synchonized (对象) {
        ……
}
```

```
synchonized 方法名{
        ……
}
```

对于同步方法，该方法所在类的实例this就是同步监视器，所以可以在同步方法中直接调用这3个方法，如this.wait()直接简写为wait()。

11.6.1 wait()和notify()方法

1. wait()方法

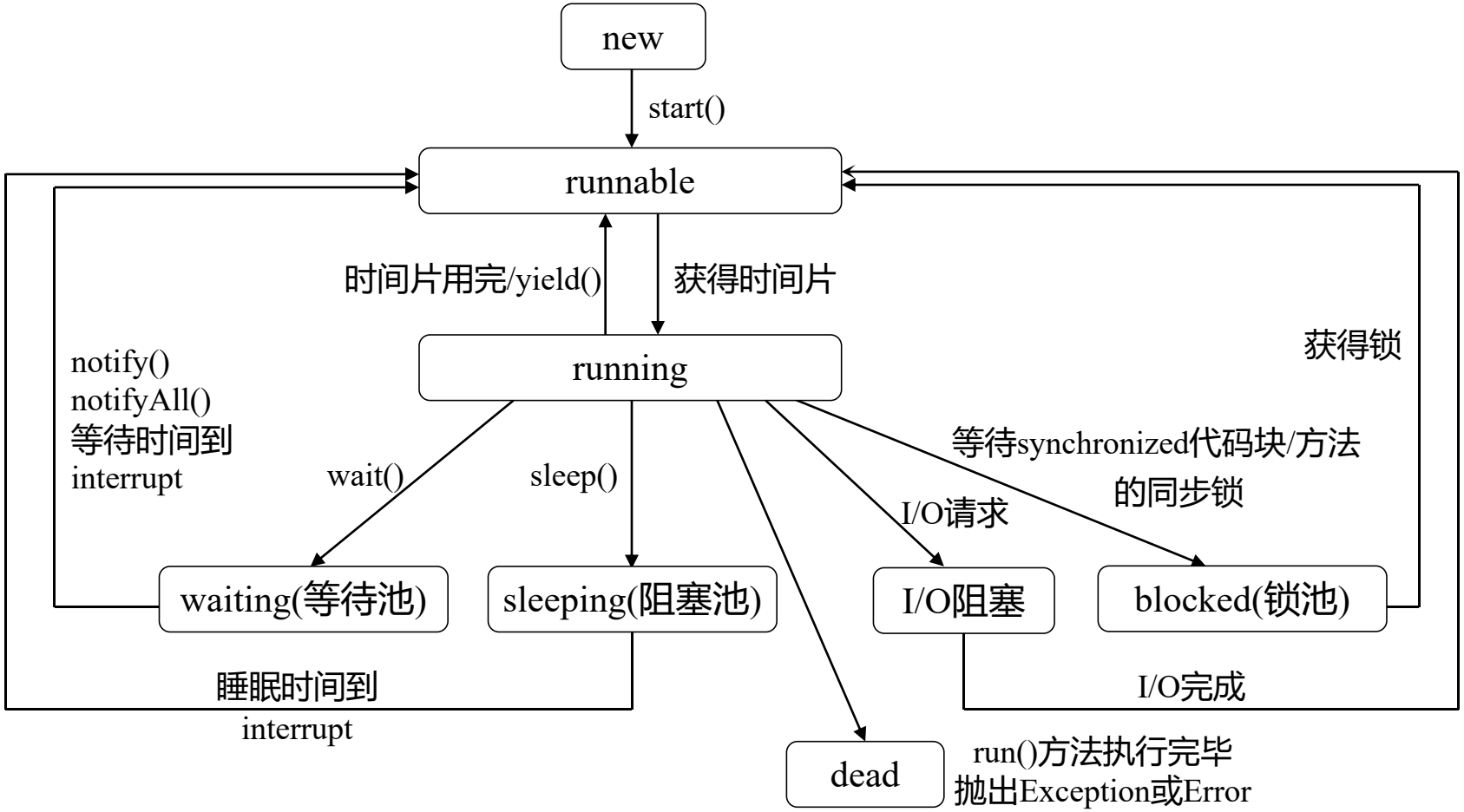
- 在线程已获得对象锁的情形下，如果该线程需要再满足一些条件才能继续执行线程任务，此时该线程可调用wait()方法进入等待池（阻塞状态的一种）。
- 线程调用wait()方法会解除对象的锁，让出CPU资源，并使该线程处于等待状态，使其它线程可以获取该对象的锁，执行该对象的同步代码块或方法。
  - void wait()：线程会一直等待，直到其他线程调用同步监视器的notify()或notifyAll()方法后苏醒；
  - void wait(long timeout)，wait(long timeout,int nanos)：方法指定了等待时间，所以如果线程在等待时间内没有被同步监视器的notify()方法唤醒，则在等待指定时间后自动苏醒。

2. notify()和notifyAll()方法

- notify()方法唤醒一个处于等待状态的线程，使之进入runnable状态。
- 某个线程执行完同步代码，或该线程使另一个线程所等待的条件得到满足，这时它利用同步监视器调用notify()方法，以唤醒一个因该同步监视器而处于等待状态的线程再次进入runnable状态。从等待状态进入runnable状态的线程，将再次尝试获得同步监视器的锁。
- notifyAll()方法：使因该同步监视器而处于等待状态的全部线程进入runnable状态。

编程时应该用notifyAll()取代notify()，因为用notify()只是从等待池释放出一个线程，至于是哪一个由线程调度器决定，是不可保证的。

线程状态转换图


new
start()
runnable
时间片用完/yield()
获得时间片
running
notify()
notifyAll()
等待时间到
interrupt
wait()
sleep()
获得锁
等待synchronized代码块/方法
的同步锁
I/O请求
waiting(等待池)
sleeping(阻塞池)
I/O阻塞
blocked(锁池)
睡眠时间到
interrupt
I/O完成
dead
run()方法执行完毕
抛出Exception或Error

11.6.1 wait()和notify()方法

【例11-8】线程间的通信。写两个线程，线程A“做”10个披萨，线程B“做”20份意大利面，要求线程A每做一个披萨，就通知线程B去做两份意大利面，线程B完成两份意大利面后通知线程A继续做披萨……。

关键问题：找到两个线程中互相向对方通信的对象。

- 两个线程的wait()和notifyAll()方法需要使用同一个同步监视器。
- 同步监视器应被两个线程共享。
- 每个线程都引入这个对象，利用构造方法对其初始化。

11.6.2 消费者和生产者模型

- 【问题描述】生产者和消费者在同一时间段内共用同一个缓冲区（仓库），生产者向缓冲区存放数据，而消费者从缓冲区取用数据。
  - 问题的关键：保证生产者不会在缓冲区满时再加入数据，消费者也不会在缓冲区空时再消耗数据。

11.6.2 消费者和生产者模型

- 【问题描述】生产者和消费者在同一时间段内共用同一个缓冲区（仓库），生产者向缓冲区存放数据，而消费者从缓冲区取用数据。
  - 问题的关键：保证生产者不会在缓冲区满时再加入数据，消费者也不会在缓冲区空时再消耗数据。